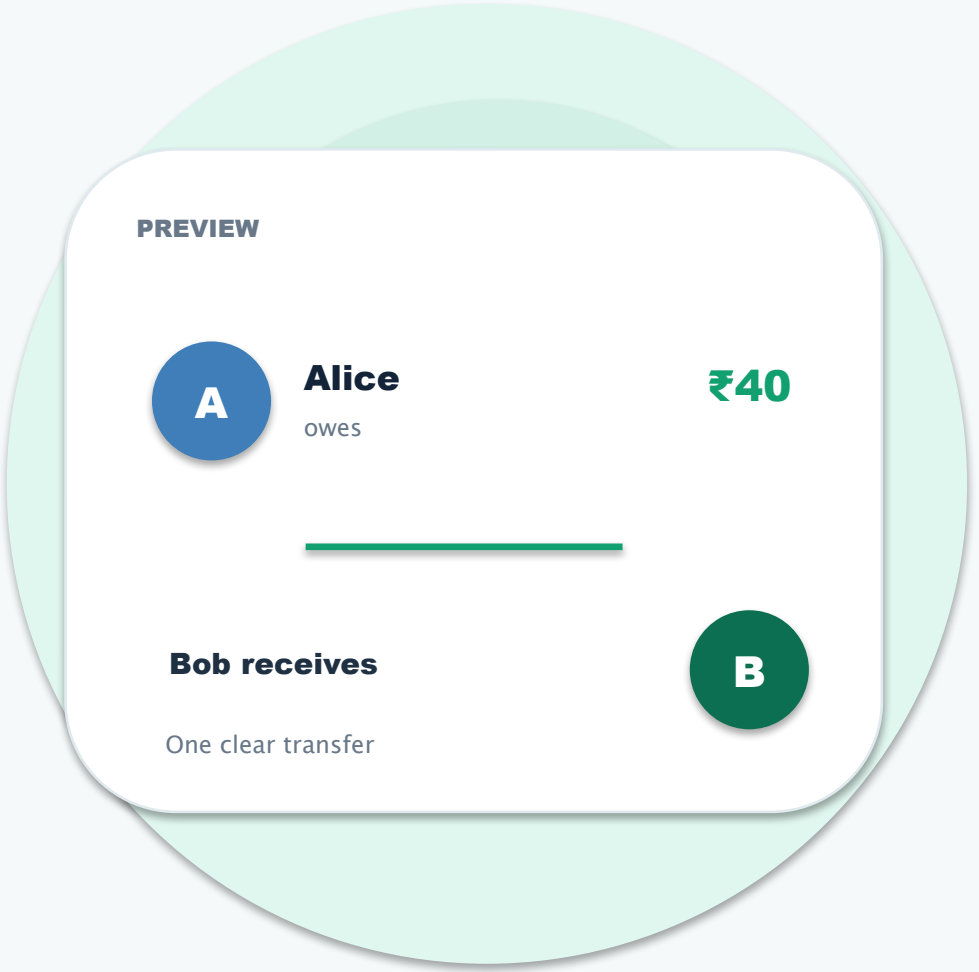
FINTECH • WEB APP

MinSettle

Split expenses.
Settle smarter.

A group expense platform that calculates fair balances and reduces the number of peer-to-peer payments needed to settle them.



Team Name : Stonks
Members : Kaushiq Srikanth S S, Litheshwaran S, Karthikraj K

01 / THE PROBLEM

Group spending creates messy IOUs

When everyone pays at different times, the group ledger becomes hard to reconcile.

Scattered payments

One person pays for food, another covers travel, and someone else handles utilities.

Manual reconciliation

Members calculate who owes whom across multiple expenses and split rules.

Too many transfers

Pairwise repayments can create unnecessary payment steps and confusion.

THE CORE QUESTION

Can we preserve every member’s net position while reducing the number of payments?

MinSettle treats the group as one shared ledger not a collection of disconnected IOUs.

One shared ledger. A clearer way to settle.

MinSettle connects expense capture, balance calculation, debt optimisation and UPI hand-off.

01

Create a group

Invite members with a shareable invite code.

02

Log expenses

Record payer, amount, category and split method.

03

Calculate balances

Aggregate paid amounts, shares and completed settlements.

04

Simplify debts

Generate a compact set of debtor → creditor transfers.

05

Pay via UPI

Open a UPI app with recipient and amount pre-filled.

Outcome: fewer settlement instructions, with each member's net balance preserved.

Built around the complete group-expense loop

Capabilities below are grounded in the repository README, frontend and backend implementation.

Group management

Create groups, join via invite code, manage members and group settings.

Flexible expense splits

Equal, percentage and custom split methods; category and recurring/one-off metadata.

Balance visibility

Per-member paid, owed and net balance breakdowns.

Debt simplification

Optimal subset partitioning for up to 8 non-zero balances; greedy fallback beyond that.

UPI hand-off

Generate UPI payment URIs to prefill payee and amount in compatible apps.

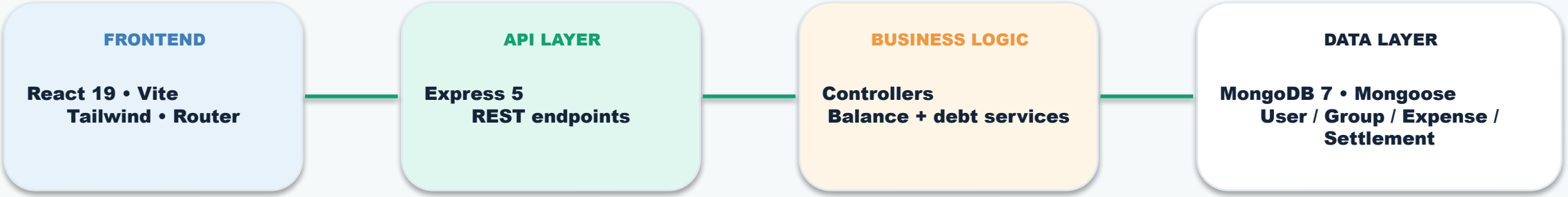
Settlement lifecycle

Pending, paid and stale settlement states; payer-restricted confirmation flow.

Important distinction: UPI deep-linking starts a payment in an external app; it is not direct bank or payment-gateway verification.

A React client backed by an Express API and MongoDB

The repository uses a modular frontend/backend structure with service-level balance and settlement logic.



Authentication

JWT-protected API routes; bcryptjs password hashing.

Precision

Amounts and splits are stored as integer paise.

Packaging

Docker Compose includes MongoDB, backend and Nginx-served frontend.

Turn expense history into each member’s net position

The balance engine aggregates what each person paid, their allocated share, and paid settlements.

TOTAL PAID

+ ₹

−

TOTAL OWED / SHARE

₹

±

SETTLEMENT ADJUSTMENTS

Paid transfers

NET BALANCE = PAID − SHARE ± PAID SETTLEMENT ADJUSTMENTS

INTERPRETATION

Positive balance

Member is a net creditor — others owe them.

Negative balance

Member is a net debtor — they owe the group.

Zero balance

Member has nothing left to settle.

Search for zero-sum groups, then generate transfers

For up to 8 non-zero balances, the code uses bitmask dynamic programming to maximise zero-sum partitions.

1. Normalise balances

Remove zero balances; retain integer paise values.

2. Find zero-sum subsets

Bitmask DP searches disjoint subsets whose balances sum to zero.

3. Settle each subset

Greedy matching inside each partition produces transfers.

REPOSITORY TEST CASE

Net balances

A	B	C	D	E
−₹40	−₹30	−₹20	+₹50	+₹40

Greedy heuristic **4 transfers**

Optimal partitioning **3 transfers**

One transfer saved in this tested case with all net balances still settling to zero.

Implemented end-to-end, with test coverage for the core maths

The repository includes the UI, API, persistence models, debt engine, tests and container setup.

Frontend

React 19 + Vite
Tailwind CSS 4
React Router 7 + Axios
Dashboard, groups, expenses, settlements

Backend

Node.js + Express 5
Mongoose 9 + MongoDB
JWT authentication + bcryptjs
REST controllers and services

Delivery

Docker Compose
MongoDB 7 container
Backend API on :5000
Frontend served through Nginx on :3000

TESTS IN REPOSITORY

- Hand-computed 3- and 4-member balance examples
- Greedy vs. optimal comparison (4 → 3 transfers)
- Greedy fallback when non-zero members exceed 8
- Zero balances, single debtor/creditor and integer-paise precision

Make group repayments simpler and more trustworthy.

MinSettle already demonstrates the full expense-to-settlement workflow; the next step is stronger financial integrity.

WHAT WORKS TODAY

- **Shared ledger:** group expenses with equal, percentage and custom splits.
- **Balance engine:** integer-paise net calculations and settlement adjustments.
- **Optimiser:** exact minimum transfer count within the 8-member non-zero cap.
- **Payment hand-off:** UPI URI generation for compatible payment apps.

NEXT ITERATION

- **Payment verification:** integrate trusted provider/bank confirmation; avoid treating a tap as proof.
- **Ledger integrity:** atomic settlement generation and stronger duplicate/current-debt checks.
- **Access control:** tighten group-admin permissions and member-removal safeguards.
- **Quality & scale:** integration tests, rate limits, request validation and observability.

MinSettle **Simplify debts. Minimise transactions.**